

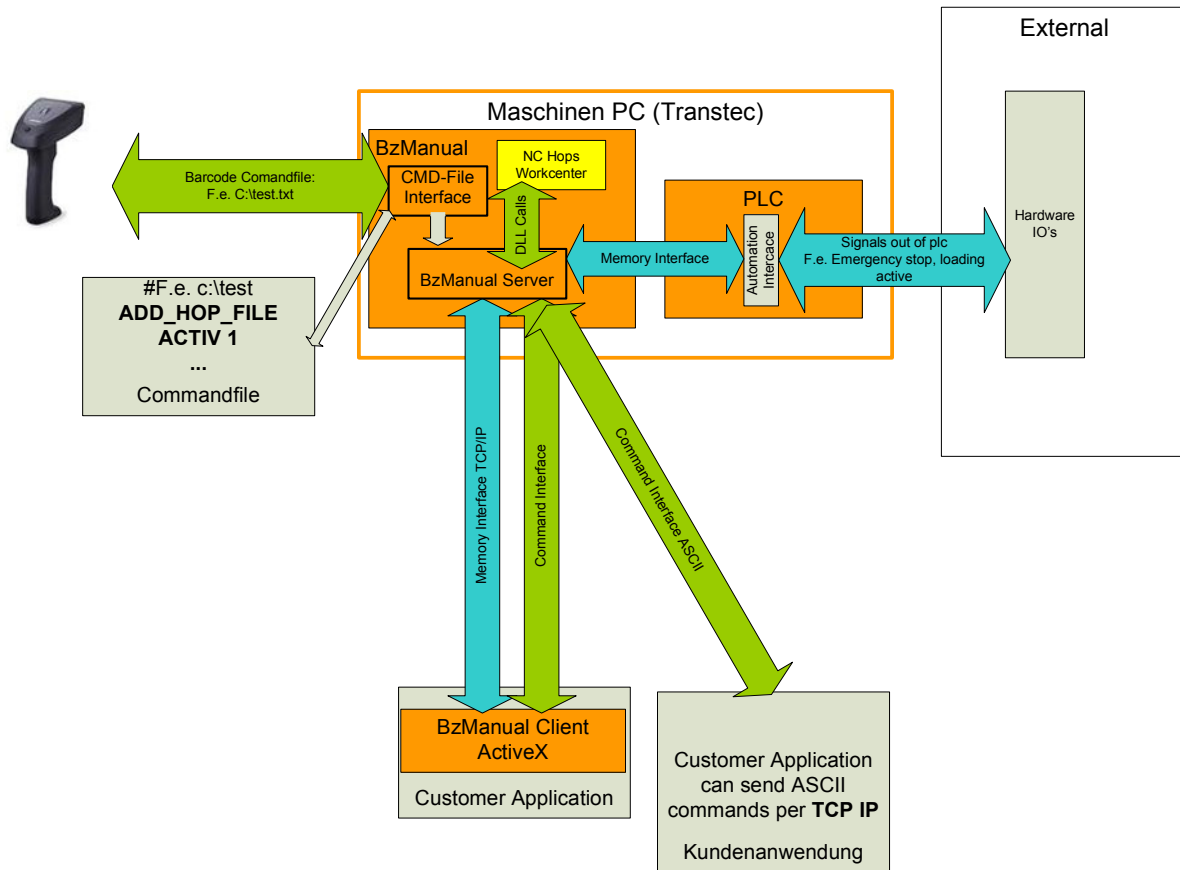
Automation Interface / TCP BZM Server

1. Inhalt

1.	Inhalt	2
2.	Overview	3
3.	TCP Server	3
3.1.	Using activeX	3
3.2.	Test Application.....	3
4.	Barcode CMD Interface	4
5.	Summary of the Commands.....	5
5.1.	Add Hopfile.....	5
5.2.	Remove Hopfile.....	6
5.3.	Activate table.....	6
5.4.	Get Hop File	6
5.5.	Get Status of table	7
5.6.	Write variables	7
5.7.	Roatate Hopfile	8
5.8.	Mirror Hopfile.....	8
5.9.	Save Machineload.....	9
5.10.	Load Machineload	9
5.11.	Load joblist	10
5.12.	End jobliste Mode	10
5.13.	Set Laser Mode	11
5.14.	Backup.....	11
5.15.	Restore	11
6.	PLC Automation interface (only Beckhoff controlled!).....	11
6.1.	SET Functions.....	12
6.1.1.	SET_BIT	12
6.1.2.	SET_BYTE.....	12
6.1.3.	SET_INT	13
6.1.4.	SET_FLOAT	13
6.2.	GET-Functions	13
6.2.1.	GET_BIT	13
6.2.2.	GET_BYTE	14
6.2.3.	GET_INT.....	15
6.2.4.	GET_FLOAT	15
7.	Diagnosis	15

2. Overview

There are two interfaces to set automatic commands for BzManual. You can use TCP Interface or a HHCMMD file for triggering commands.



3. TCP Server

3.1. Using activeX

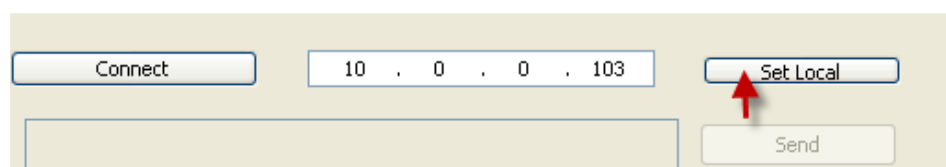
The ActiveX is placed in `d:\programs\controls\BZManualX.ocx`. If you include this ActiveX in your on application you can use the documented functions to remote control the BzManual.

3.2. Test Application

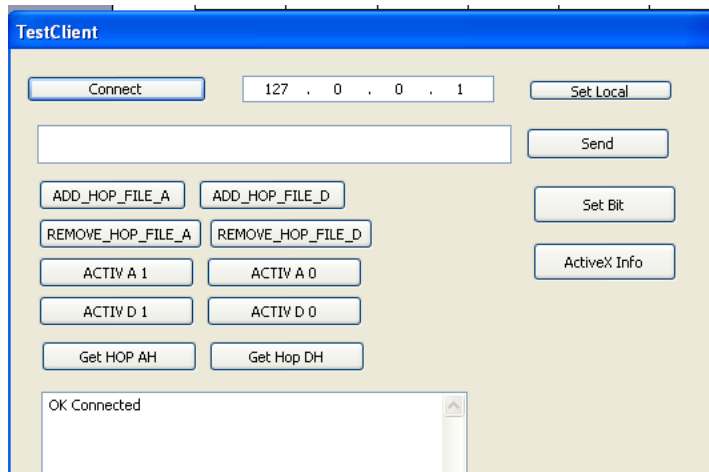
`d:\Programs\BZManual\Apps\AutomationInterface\TestClient.exe`

This is a Test-Application to try your remote commands.

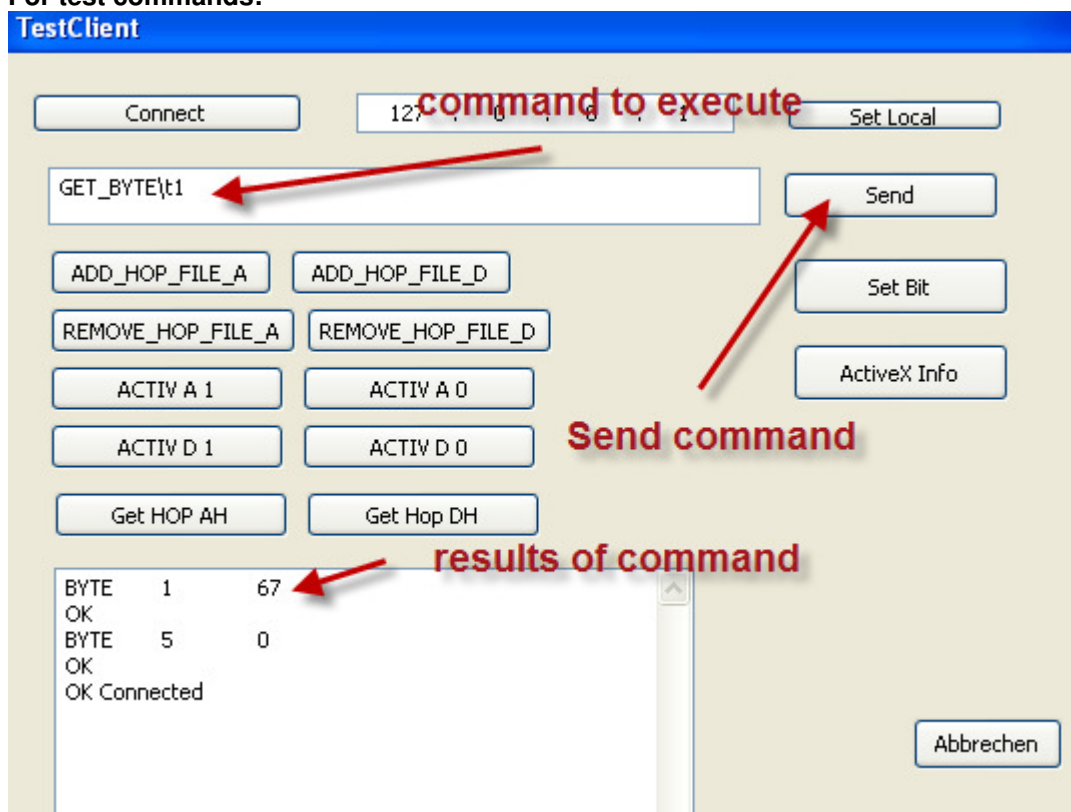
- Set Local Button for set local ip address to use application on some pc BzManual is running.
- Otherwise set IP address of pc the BzManual with automation interface is running.



→ Then press connect button. If the connection is successful the execution dialog is getting active.



For test commands:



4. Barcode CMD Interface

For using barcode cmd-line interface you have to define a command file. This file can have one command each line. You can also have a comment starting with #.

The triggered start to begin doing the commands is the scan of a special barcode. This barcode should begin with the word **[hhcmd=]**. It should follow the filename which includes the commands.



Hhcmd=C:\test.txt



Hhcmd=d:\Programs\BZManual\BarCodeCmd.txt

EXAMPLE BarCodeCmd.txt:

```
# make Part1 last changed on 2011
# This is comment
ADD_HOP_FILE;AH;d:\CAMPUS\Data\Hops\Hop\F1.hop;3
# Change Size of this part.
WRITEVAR;AH;DX;50
WRITEVAR;AH;DY;50
# Activate table
ACTIV;A;1
```

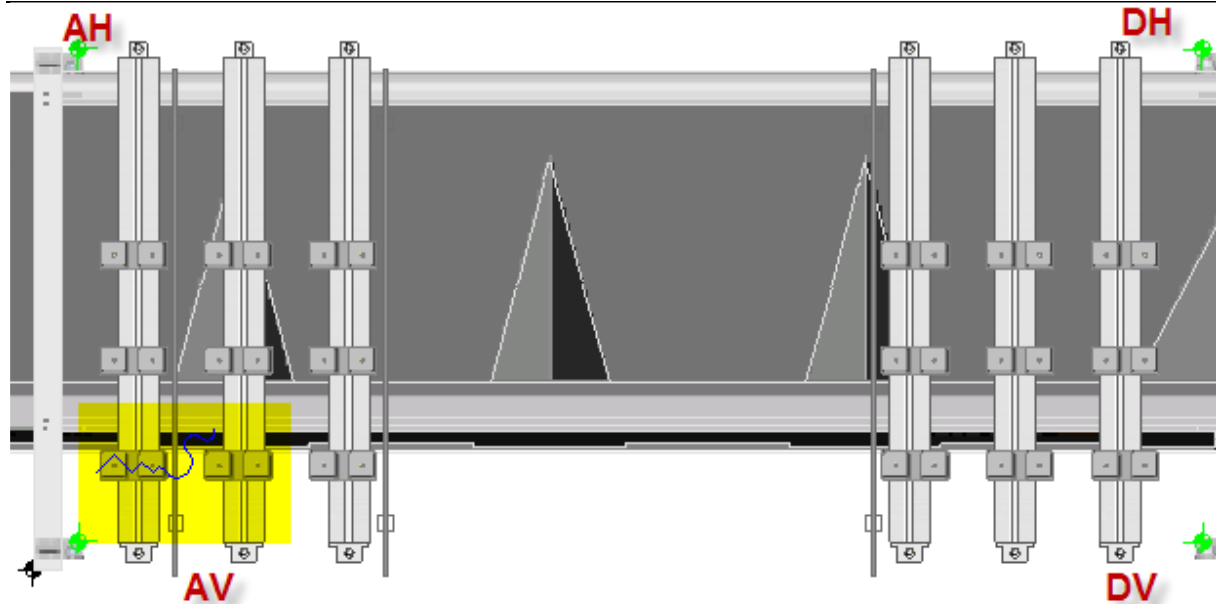
5. Summary of the Commands

5.1. Add Hopfile

Add one hop file on a file.

1. Zero points identifier (f.e. DV/ AV/DH/AV)
2. Path and filename to put hop file.
3. Number of parts from this file to produced (count).

TCP	ADD_HOP_FILE\tdV\td:\CAMPUS\Data\Hops\Hop\Ecke.hop\t5
HHCMD	ADD_HOP_FILE;DV;d:\CAMPUS\Data\Hops\Hop\Ecke.hop;5
OCX	long TcpAddHopFile(LPCTSTR Edge, LPCTSTR Filename, long count);



5.2. Remove Hopfile

Removes the hop file on the given table edge (Zeropoints).

1. Zero points identifier (f.e. DV/ AV/DH/AV)

TCP	REMOVE_HOP_FILE\tAH
HHCMD	REMOVE_HOP_FILE;AH
OCX	<code>long TcpRemoveHopFile(LPCTSTR Edge);</code>

5.3. Activate table

Activate a table. If there is a part on the chosen edge, the postprocessor will work with the file.



1. Table to Activate or deactivate (A or D).
2. State to set for this table (0 – deactivate, 1 - active)

TCP	ACTIV\tA\t1
HHCMD	ACTIV;A;1
OCX	<code>long TcpActive(LPCTSTR Table, long State);</code>

5.4. Get Hop File

Get the Hopfile from a field.

1. Table to Activate or deactivate (A or D).

TCP	GET_HOP_FILE\τAH
HHCMD	GET_HOP_FILE;AH
OCX	Long GetHopFile(LPCTSTR Edge);

Return Values:

No file on filed:

GET_HOP_FILE\τAH\τNOFILE

Hopfile on filed:

GET_HOP_FILE AV D:\CAMPUS\Data\Hops\Hop\LASER_Test_Daten\Test1.hop

5.5. Get Status of table

Get the actual status of a table.

1. Table to get status

TCP	GET_STATUS\τA
HHCMD	GET_STATUS;A
OCX	

Return Values:

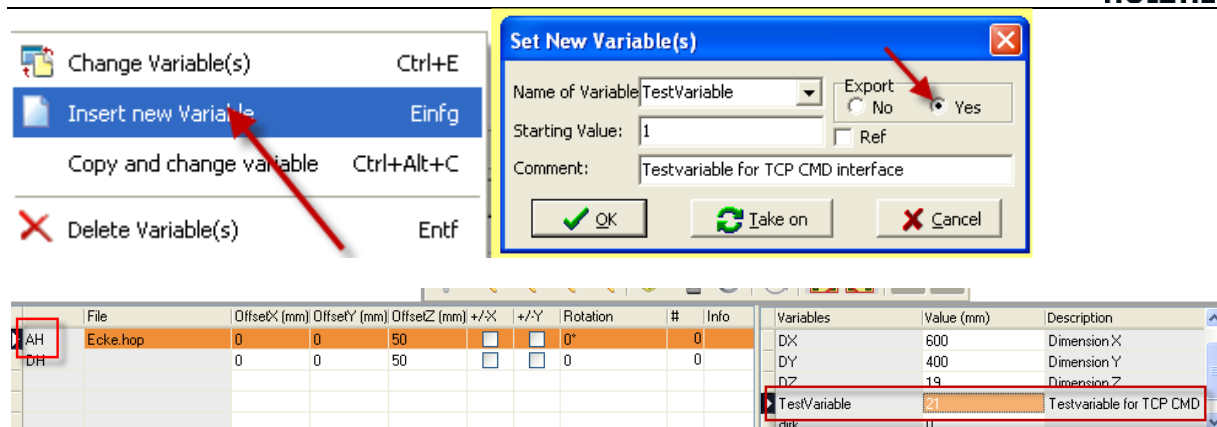
	BIT 7	BIT 6		BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
	Tisch Start gespeichert ¹	Tisch Aktiv		Tisch angefordert ¹	Tisch abgebrochen ¹	Tisch fertig ¹	Tisch läuft ¹	Tisch bereit ¹

5.6. Write variables

This Function writes variables of a hopfile which it is added on zero point but not activated yet. The variables have to be defined as external in Hops. Every parameter viewed in table down left can be written with this function.

1. Zero points identifier (f.e. DV/ AV/DH/AV).
2. Name of Variable which is exported out of the Hops.
3. Value for the variable to write.

¹ Fehler! Verweisquelle konnte nicht gefunden werden.



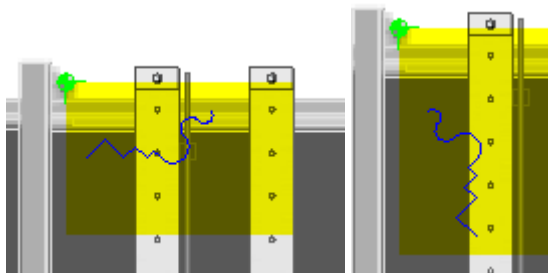
!ATTENTION: Write variables before activate a hop file.

TCP	WRITEVAR\tAH\tTestVariable\t21
HHCMD	WRITEVAR;AH;TestVariable\;21
OCX	Long TcpWriteVar(LPCTSTR Table,LPCTSTR Variable ,double Value)

5.7. Roatate Hopfile

This function rotates the part on zerpoint is given out of parameters with quarter circle part.

1. Zero points identifier (f.e. DV/ AV/DH/AV).
2. Level / degrees of the roatation. 1- 90 °,2 – 180 °, 3 – 270 °



!ATTENTION: rotate before activate a hop file.

TCP	ROTATE\tAH\t3
HHCMD	ROTATE;AH;1
OCX	Long TcpRotateHopfile(LPCTSTR Table,double Value)

5.8. Mirror Hopfile

Mirrors the part is applied on the paramteriezed zerpopoint.

1. Zero points identifier (f.e. DV/ AV/DH/AV).
2. Way to mirror in 1 - X, 2 – Y, 3 – X/Y

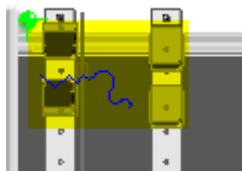


TCP	MIRROR\TAH\3
HHCMD	MIRROR;AH;3
OCX	Long TcpMirrorHopfile(LPCTSTR Table,double Value)

5.9. Save Machineload

Saves a actual assignment of parts (hop files), pads and consoles of the complete machine.

1. Save filename with ending jlx to be compatible to workcenter.



TCP	SAVE_MACHINELoad\tc:\test.jlx
HHCMD	SAVE_MACHINELoad;c:\test.jlx
OCX	Long TcpSaveMachineload(LPCTSTR Filename)

5.10. Load Machineload

Load a complete assignment of consoles, pads and work pieces on the machine.

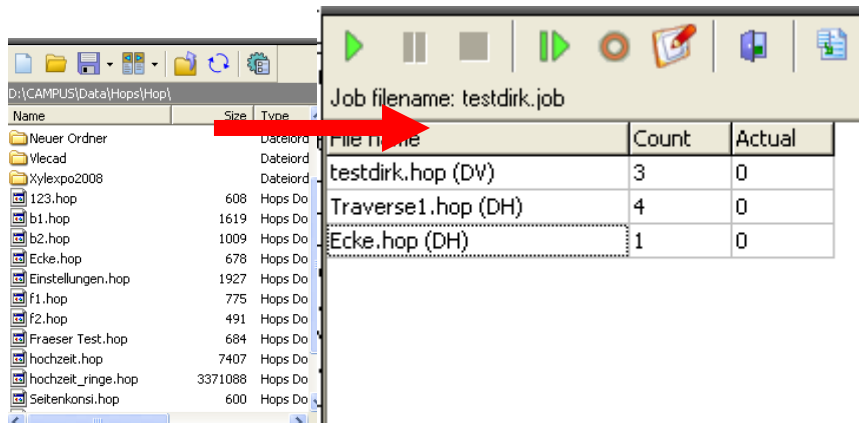
1. Loading filename with ending jlx to be compatible to workcenter.

TCP	LOAD_MACHINELoad\tc:\test.jlx
HHCMD	LOAD_MACHINELoad;c:\test.jlx
OCX	Long TcpLoadMachineload(LPCTSTR Filename)

5.11. Load joblist

Activate and load a joblist in workcenter.

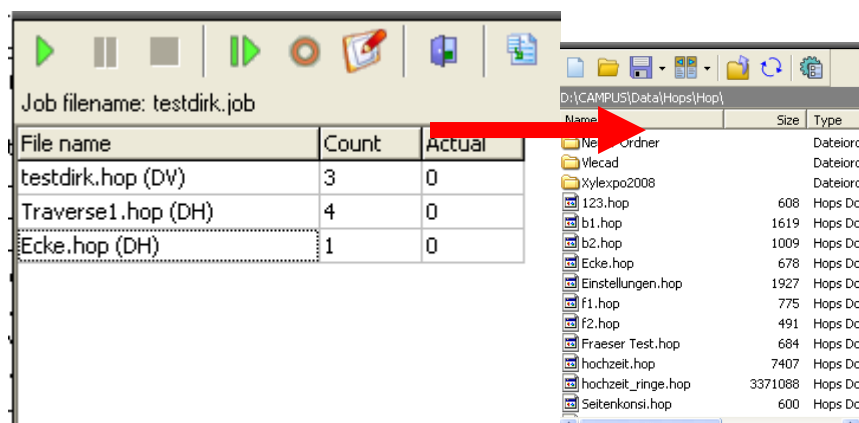
1. Loading filename with ending job as joblist.



TCP	LOADJOBLIST\td:\CAMPUS\Data\Jobs\testdirk.job
HHCMD	LOADJOBLIST;d:\CAMPUS\Data\Jobs\testdirk.job
OCX	Long TcpLoadMachineload(LPCTSTR Filename)

5.12. End jobliste Mode

End the Joblist mode in workcenter

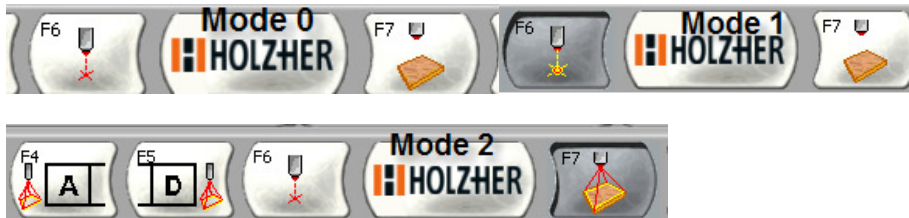


TCP	ENDJOBLIST
HHCMD	ENDJOBLIST
OCX	TcpSendString(LPCTSTR Str= "ENDJOBLIST")

5.13. Set Laser Mode

Set the active lasermode.

1. Laser Mode 0 – no Laser, 1 – point laser, 2- 3D-Laser, 3 – Both Point and 3D-Laser



!ATTENTION: set laser mode before activate a hop file.

TCP	SET_LASER_MODE\t2
HHCMD	SET_LASER_MODE;3
OCX	long CBzmanualXCtrl::TcpSetLaserMode(long State)

5.14. Backup

Starts the workcenter backup dialog of BzManual.

TCP	BACKUP
HHCMD	BACKUP
OCX	TcpSendString(LPCTSTR Str= "BACKUP")

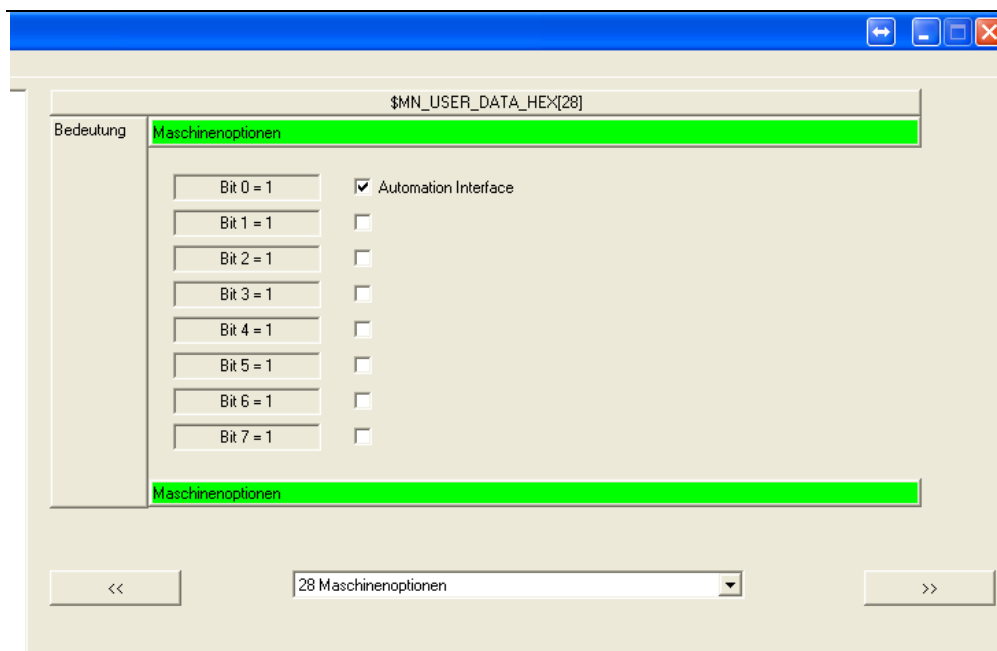
5.15. Restore

Starts the workcenter restore dialog of BzManual.

TCP	RESTORE
HHCMD	RESTORE
OCX	TcpSendString(LPCTSTR Str= "RESTORE")

6. PLC Automation interface (only Beckhoff controlled!)

This is the documentation of the PLC-Automation interface for bechhoff controlled HOLZ-HER BAZ. To use the interface the Automation Interface bit in MDConfig should me activated.



6.1. SET Functions

The SET-Functions writes data into the PLC-Interface.

6.1.1. SET_BIT

TCP	SET_BIT\t8\t1
HHCMD	SET_BIT;8;1
OCX	TcpSendString(LPCTSTR Str= "SET_BIT\t8\t1")

BIT	Function

6.1.2. SET_BYTE

TCP	SET_BYTE\t8\t14
HHCMD	SET_BYTE;8;14
OCX	TcpSendString(LPCTSTR Str= "SET_BYTE\t8\t14")

Byte	Bit	Function
Field A		
10	0	Start the Program.
10	6	Vacuum on.

Field D		
15	0	Start the Program
15	6	Vacuum on

6.1.3. SET_INT

TCP	SET_BIT\t8\t1
HHCMD	SET_BIT;8;1
OCX	TcpSendString(LPCTSTR Str= "SET_BIT\t8\t1")

INT	Function

6.1.4. SET_FLOAT

TCP	SET_BIT\t8\t1
HHCMD	SET_BIT;8;1
OCX	TcpSendString(LPCTSTR Str= "SET_BIT\t8\t1")

FLOAT	Function

6.2. GET-Functions

The GET-Functions gets variables from the PLC. Every change of an value of the PLC output interface will be send as tcp message to all connected clients.

6.2.1. GET_BIT

Get the bit information out of the automation interface.

1. Number of bit to read. Values 0..31

TCP	GET_BIT\t8
HHCMD	GET_BIT;8
OCX	TcpSendString(LPCTSTR Str= "GET_BIT")

BIT	Function

6.2.2. GET_BYTE

Get byte status information from PLC.

1. Number of byte to read. Values 0..31

TCP	GET_BYTE\t30
HHCMD	GET_BYTE;30
OCX	TcpSendString(LPCTSTR Str= "GET_BYTE\t30")

Byte	Bit	Function
1	0	Machine has no emergency stop active. TRUE – no emergency, FALSE – emergency active.
1	1	Machine switched on. TRUE – switched on, FALSE – not switched on.
1	2	Machine is ready to work. All drives ready and machine is referenced. TRUE – ready, FALSE – not ready.
1	3	Machine is in error. TRUE – Machine is in error state, FALSE – no error.
1	5	Machine is in automatic program mode. TRUE – automatic program mode active, FALSE – not.
1	6	Machine is in manual mode. Axis can be moved with BHG. TRUE – manual mode, FALSE – not.
1	7	Reset signal pulse.
2	0	Machine filed are in linked mode (A and D linked).
3	0	Machine waits for user acknowledge. F.e. Tool change, start reference or safety mat.
3	1	Milling spindle and drilling head is not moving. TRUE – stopped, FALSE – is moving.
3	2	Axes in cnc channel are moving. TRUE – is moving, FALSE – no axes moves.
3	3	Movement stop active. No movement allowed signal. TRUE – not allowed, FALSE – normal mode.
Field A		
10	0	Field is active and program is ready to start.
10	1	Field is working. Program is running.
10	2	Field ready. Program ended.
10	3	Field canceled.
10	6	Field vacuum is on
10	7	Field vacuum is off
12	0	Field pins are retracted.
12	1	Field pins are extended.
Field D		
15	0	Field is active and program is ready to start.
15	1	Field is working. Program is running.
15	2	Field ready. Program ended.
15	3	Field canceled.
15	6	Field vacuum is on
15	7	Field vacuum is off
17	0	Field pins are retracted.
17	1	Field pins are extended.

6.2.3. GET_INT

TCP	GET_INT\t5
HHCMD	GET_INT;5
OCX	TcpSendString(LPCTSTR Str= "GET_INT\t5")

INT	Function

6.2.4. GET_FLOAT

TCP	GET_FLOAT\t2
HHCMD	GET_FLOAT;2
OCX	TcpSendString(LPCTSTR Str= "GET_FLOAT\t2")

FLOAT	Function

7. Diagnosis

For diagnosis to check which commands are executed you can use the DDE- Debug window in BzManual.

